

Module Summary – AI Programming Foundations

Nina Haide | May 6th, 2026

1. Overview

This report documents the design, implementation and findings of the AI Career Compass project, built as the capstone deliverable for Module 2 of the Udacity Master in AI programme. The project consists of a Python-based data workflow implemented in a Jupyter Notebook that ingests, cleans, analyses and visualises live job posting data from the Adzuna Jobs API. The dataset Adzuna DACH AI Job Postings, scraped on May 5th, 2026 and available at <https://developer.adzuna.com> - covers 442 unique AI-related job postings across Germany, Austria and Switzerland.

2. Dataset Description

The dataset was collected programmatically via the Adzuna REST API using a custom Python script (`fetch_ai_jobs.py`) and comprises 442 rows and 15 columns. Each row represents a single job posting and includes fields for job title, company, location, country, Adzuna category, contract type, contract time, salary minimum, salary maximum, posting date, a truncated description excerpt, a researcher-assigned role category, the search query that surfaced the posting and a redirect URL. The five role categories - AI Engineering, AI Consulting, AI Product/Project Management, AI Integration and Data Analytics - were defined a priori based on the researcher's professional interest in hybrid technical-business AI roles. Key variables of analytical focus were `role_category`, `country`, `location`, `salary_min`, and `salary_max`; the latter two were sparse, with only 18 of 442 postings disclosing compensation figures.

3. Workflow Description

Ingestion. Job postings were fetched via paginated API calls across 31 search queries and three country endpoints (DE, AT, CH). Results were deduplicated by posting ID and consolidated into a single CSV file. The search query used to retrieve each posting was retained as a column to support transparency and downstream bias analysis.

Cleaning. Two cleaning functions were applied. `clean_job_titles()` stripped German gender notation suffixes (e.g. (m/w/d)), em-dashes and reference numbers from job titles, producing a normalised `title_clean` column. `clean_salary_data()` flagged implausible salary values below €500 as likely hourly rates, nulled them out, added a boolean `salary_available` flag, and computed a `salary_midpoint` column as the mean of min and max where both were present.

Exploratory Analysis. Three EDA functions were implemented: `summarise_dataset()` produced shape, missing value rates and categorical distributions; `analyse_by_group()` generated crosstabs of role category by country and salary availability by group; and `check_correlations()` computed a correlation matrix on encoded categorical variables alongside a salary breakdown by role and country.

Visualisations. Three labelled plots were produced using Matplotlib and Seaborn (see Section 5). Each was saved as a PNG file alongside the notebook for reference in this report.

Summary. Findings were documented in a Markdown cell within the notebook, covering key patterns, limitations, and surprising findings.

4. Key Decisions and Assumptions

Cleaning choices. The decision to null out rather than impute salary values flagged as hourly was deliberate. Imputing values from a sample of only 18 observations would introduce more uncertainty than it resolves, and transparent handling of missing data is a core principle of reproducible data science practice (Danchev, 2022). The `salary_available` flag was added rather than simply dropping rows, preserving the full dataset for non-salary analyses while clearly marking the boundary of what can and cannot be concluded from compensation data.

EDA focus. The exploratory analysis deliberately prioritised structural patterns - role distribution, country breakdown, and salary availability - over deeper statistical modelling, which is appropriate for a dataset of 442 rows with sparse numeric fields. As Danchev (2022) argues, exploratory analysis should function as a transparent description of data features before any modelling decisions are made, enabling readers to evaluate the data's fitness for purpose.

Visualisation choices. Figure 1 (stacked bar chart) was designed to show both absolute volume and country composition simultaneously, making the Germany dominance visible at a glance. Figure 2 (box and strip plot) overlays individual data points on the box to acknowledge the very small salary sample - a best practice when n is too low for

a box plot alone to be meaningful (Waskom, 2021). Figure 3 (horizontal bar chart) uses a horizontal layout specifically to accommodate the length of German location strings without truncation.

5. Results and Interpretation

Figure 1 reveals that AI Engineering (131 postings) and Data Analytics (108) are the most advertised AI role categories in the DACH region, followed by AI PM (98), AI Integration (82) and AI Consulting (23). Germany accounts for 82% of all postings, confirming its dominance as the primary AI hiring market in the DACH region. Switzerland and Austria contribute 10% and 8% respectively.

Figure 2 shows that among the 17 postings with usable salary data (after excluding the Switzerland CHF outlier of 416,000), AI Engineering roles cluster between €65,000 and €105,000, while Data Analytics roles show higher midpoints despite a smaller sample. The wide interquartile range in the AI PM category reflects genuine heterogeneity in compensation for product and project management roles across seniority levels.

Figure 3 identifies Berlin, Munich and Frankfurt as the primary AI hiring hubs in Germany, with Vienna and Zurich leading in Austria and Switzerland respectively. The high volume of unspecified 'Deutschland' and 'Schweiz' locations suggests that a significant share of postings are either fully remote or were posted without a specific city, which limits location-based analysis.

The correlation matrix from the EDA revealed a moderate positive relationship ($r = 0.52$) between country encoding and salary midpoint, driven by Switzerland's single high-value disclosure. This finding should be interpreted cautiously given the small salary sample.

6. Responsible Practice (Bias and Data Quality)

Several sources of potential bias are present in this dataset and should be acknowledged. First, the keyword-based API collection method introduces selection bias: only postings that match the pre-defined search terms are included, meaning roles with non-standard titles or descriptions may be systematically excluded. This is particularly relevant for AI Consulting roles, which are underrepresented ($n=23$) and likely appear under titles such as 'Digitalisierungsberater' or 'Management Consultant' that were not in the query set.

Second, the near-total absence of salary data (96% missing) introduces disclosure bias: employers who disclose salaries may differ systematically from those who do not - for example, being more likely to be international companies, startups, or roles in competitive talent markets. Any salary conclusions therefore reflect a self-selected subsample rather than the full market. This aligns with documented patterns of salary opacity in European job markets (Danchev, 2022).

Third, the single-point-in-time collection means the dataset cannot distinguish between structural market conditions and transient hiring fluctuations. To reduce these risks in future iterations, the data collection could be extended across multiple time periods, search terms could be expanded to include German-language equivalents and salary data could be supplemented from survey-based sources such as the Levels.fyi or Stack Overflow Developer Survey datasets.

7. Reproducibility

This project was designed with reproducibility as a first-class concern. As Danchev (2022) notes, Jupyter Notebook fosters reproducible workflow by integrating research questions, data inputs, code, documentation and narrative text in a single document - a principle sometimes called 'literate programming' (Knuth, 1984, as cited in Danchev, 2022). All code in this project follows this model: every analytical decision is documented in Markdown cells adjacent to the code that implements it.

To reproduce the full workflow: (1) clone the GitHub repository at <https://github.com/n1n4ch/ai-programming-foundations-project>; (2) install dependencies using `pip install -r requirements.txt`; (3) optionally re-run `fetch_ai_jobs.py` to refresh the dataset (note: results will differ slightly from the May 2026 snapshot as Adzuna listings change daily); and (4) open `data_workflow.ipynb` and run all cells via Kernel → Restart & Run All.

The repository uses a branching Git workflow with a dedicated development branch for iterative work and regular commits documenting each stage of the project. The `requirements.txt` file was generated via `pip freeze` to pin exact package versions, minimising environment-related reproducibility failures (Danchev, 2022).

8. Future Directions

The future requirements of the project respective to machine learning and agentic automation will be discussed in the following.

Regarding the Machine learning component, the `role_category` column provides a clean supervised classification target predicting whether a posting belongs to Engineering, Consulting, PM, Integration or Analytics from features such as location, contract type and description keywords. The `salary_midpoint` column provides a regression target for compensation prediction once the dataset is enriched with more salary disclosures in future collection runs.

For a neural network and NLP use case the `description` column contains up to 500 characters of job posting text per row, providing a foundation for sentence embeddings, role similarity clustering and skills extraction using transformer-based models such as DistilBERT or sentence-transformers. This directly supports the deep learning and NLP work planned for Modules 4 and 5.

In view of agentic automation potential the `fetch_ai_jobs.py` script is architecturally ready to be extended into an autonomous data pipeline. An agent could be designed to run the collection script on a schedule, detect shifts in role demand or salary disclosure rates across snapshots and surface insights without manual intervention - directly aligning with the autonomous workflow design covered in Module 7. The `search_query` column retained in the dataset enables the agent to evaluate which query terms are producing the most relevant results and adjust its strategy accordingly.

9. References

Danchev, V. (2022). Reproducible Data Science with Python: An Open Learning Resource. *Journal of Open Source Education*, 5(56), 156. <https://doi.org/10.21105/jose.00156>

Waskom, M. L. (2021). Seaborn: Statistical Data Visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>

McKinney, W. (2010). Data Structures for Statistical Computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 51–56). <https://doi.org/10.25080/Majora-92bf1922-00a>

Adzuna. (2025). *Adzuna Jobs API Documentation*. <https://developer.adzuna.com>